



# Programmieren

Falk Morgenstern  
Leon Brettin

Foliensatz basierend auf Unterlagen von Prof. Huhn, Prof. Meyer und weiteren.  
Nur zur Verwendung in der Vorlesung „Programmieren“ an der Ostfalia.  
Keine Weitergabe.



# Programmieren

## 2. Nützliches aus der Java-API



# Die Java-API

- Die Java-API (Application Programming Interface) bietet mit jeder Version mehr Unterstützung für den Entwickler.
- Die Standard Klassenbibliothek
- Hier nur folgende Aspekte:
  - Wrapper-Klassen
  - Strings
  - Jars
  - Dateien, I/O (später)



# Programmieren

## 2.1. Wrapper



# Wrapper-Klassen

- Java bietet für jeden primitiven Datentyp neben der Realisierung als primitiver Datentyp spezielle Wrapper-Klassen (Mantelklassen) an. Die Wrapper-Klasse heißt (fast immer) wie der primitive Datentyp und beginnt mit einem Großbuchstaben.
- Wrapper-Objekte kapseln einen primitiven Wert in einem Objekt, sodass eine Referenz auf diesen Wert existiert.

| Wrapper-Klasse | Primitiver Datentyp |
|----------------|---------------------|
| Byte           | byte                |
| Short          | short               |
| Integer        | int                 |
| Long           | long                |
| Double         | double              |
| Float          | float               |
| Boolean        | boolean             |
| Character      | char                |



# Wrapper-Klassen

- Aufgaben:
  - Bereitstellung von statischen Hilfsmethoden zur Konvertierung eines primitiven Datentyps in einen String (Formatierung) und vom String zurück in einen primitiven Datentyp ( Parsen).
  - Java bietet generische Datenstrukturen wie Listen und Mengen. Die können nur Referenzen aufnehmen. Wenn man also eine Liste von `int` realisieren möchte, muss man `int`-Objekte schaffen können (allgemein einen Container für primitive Werte)



# Wrapper-Klassen

- Nutzen?
- Wrapper-Objekte
  - können `null` sein, denn undefiniert ist i. allg. nicht  $\emptyset$ .
  - können Exceptions werfen → einheitliche Problembehandlung.
  - bieten die `toString`-Methode zur Umwandlung in Strings.
- Java konvertiert weitgehend automatisch zwischen primitiven Datentypen und den Wrappern (boxing und unboxing), wobei die Darstellung als primitiver Datentyp bevorzugt wird (Effizienz).



# Erzeugen von Wrapper-Objekten

- Drei Möglichkeiten:
  - statische `valueOf()`-Methoden
  - Boxing: Aus einem primitiven Wert erzeugt der Compiler automatisch durch Aufruf der `valueOf()`-Methode ein Wrapper-Objekt, falls benötigt
  - über die Konstruktoren der Wrapper-Klassen (deprecated).

```
Integer intref1 = Integer.valueOf( "42" ); // valueOf()  
Long lngref1 = Long.valueOf( 0xA01 ); // valueOf()  
Integer intref2 = new Integer( 29 ); // Konstruktor  
Long lngref2 = new Long( 12345678 ); // Konstruktor  
Double dref = new Double( 12.3 ); // Konstruktor  
Boolean boolref = true; // Boxing  
Integer intref3 = 42; // Boxing
```





# Automatisches (Un)Boxing

- Java konvertiert automatisch zwischen Wrapper-Objekten und den primitiven Datentypen

```
public class WrapperBeispiel {  
    public static void main(String[] args) {  
        Double e = 2.0; Double f = 2.0;  
        double g = e;  
  
        if((g == 2.0)&&(f == 2.0)) { System.out.print("A "); }  
        if(e == f) { System.out.print("B "); }  
        if(g == e) { System.out.print("C "); }  
        Integer b = 1; Integer c = 1;  
        if(b == c) {  
            System.out.println("D");  
        }  
    }  
}
```

- Ausgabe
  - A C D
- Werte sind offenbar gleich, aber Vorsicht!



# Immutable

- Wrapper-Objekte verhalten sich nicht immer wie erwartet!
- Wrapper-Objekte werden einmal erzeugt und sind dann **unveränderbar** (**immutable**),
- Soll ein neuer Wert gewrappt werden, muss ein **neues** Wrapper-Objekt erzeugt werden,
- Zusätzlich werden für kleine ganzzahlige Werte alle Wrapper-Objekte vorgehalten und bei Bedarf benutzt - Compiler-Details (das erklärt auf der letzten Folie das D).

```
Integer io = 42; // Boxing
Integer ir = io;
io = Integer.valueOf(io + 1); // Wert von io ändern
System.out.println(io); // 43
System.out.println(ir); // 42
```





# Quiz: Boxing – new – valueOf()

```
Integer n1 = 20;  
Integer n2 = new Integer(20);  
Integer n3 = Integer.valueOf(20);  
Integer n4 = n1 + n2 - n3;  
System.out.println(n1 == n2);  
System.out.println(n1 == n3);  
System.out.println(n1 == n4);  
System.out.println(n2 == n4);
```



Die Ausgabe ist:

A: **true true true true**  
B: **false true true false**

C: Compilerfehler  
D: **false false false false**

Wenn ich die **20** überall in **2020** ändere, dann ist die Ausgabe:

A: **true true true true**  
B: **false true true false**

C: Compilerfehler  
D: **false false false false**



# Immutable Objects – wofür?

- Ein Objekt heißt unveränderbar (immutable), wenn der Zustand (Wert) nicht mehr verändert werden kann, nachdem es erzeugt wurde. Beispiele:
  - Wrapper-Objekte
  - Strings
- Unveränderbare Objekte sind sinnvoll in nebenläufigen Anwendungen.
  - Da der Zustand nicht verändert werden kann, können nebenläufig zugreifende Prozesse oder Threads solche Objekte gemeinsam nutzen, ohne sich gegenseitig zu beeinflussen oder einen inkonsistenten Zustand verursachen zu können.
- Wenn man mit unveränderbaren Objekten arbeitet, muss man für jede Wertänderung ein neues Objekt erzeugen (Overhead, aber vereinfachte Garbage Collection).
- Dafür muss man keine Vorkehrungen gegen inkonsistente Zustände treffen.



# Programmieren

## 2.2. Zeichenketten



# Klasse String

- Jeder String (=konkretes String-Objekt) ist unveränderbar (immutable).



# Klasse String

- Alle String-Literale im Quelltext eines Programms werden vom Compiler in einer Tabelle verwaltet. Mehrfache Vorkommen verweisen dann auf das gleiche Objekt – analog zur Behandlung kleiner ganzzahliger Werte
- Vorsicht: Wenn Sie die keine String-Literale(im Quelltext) nutzen, sondern über eine Datei einlesen lassen, gilt das i. Allg. nicht.
- Konsequenz: Referenzdatentypen immer mit equals vergleichen, Primitivedatentypen immer mit ==

```
String t1 = "Textbeispiel";  
String t2 = "Textbeispiel";  
if (t1 == t2) {System.out.println("Gleich");} // Ausgabe: Gleich
```





# Nützliche String-Methoden



```
String s = " Guten Morgen! ";  
//der i.te Buchstabe, beginnend ab 0  
char c = s.charAt(1); //liefert 'G'  
//schneidet ein Teilwort aus: von startindex bis (endindex -1)!  
String teil = s.substring(3, 5); //liefert "te"  
//entfernt führende und endständige Leerzeichen  
String neu = s.trim();  
int l = neu.length(); // Anzahl der Zeichen = 13  
boolean b1 = neu.startsWith("Gut"); //true  
boolean b2 = neu.endsWith("tag"); //false  
//vergleicht aber ohne Gross/Kleinschreibung  
boolean b3 = neu.equalsIgnoreCase("GUTEN MORGEN!"); //true  
//wandelt alle Gross- in Kleinbuchstaben um  
String neu1 = neu.toLowerCase();  
//ersetzt alle Vorkommen von altBuchstabe durch neuBuchstabe  
String z = neu.replace('n', 'r'); //= "Guter Morger!"
```



# Nützliche String-Methoden

```
//teilt den String am angegebenen Trennzeichen
//Teile werden in einem Array gespeichert
String[] s_arr = neu.split(" "); //= {"Guten", "Morgen!"}
//mehrere Trennzeichen: | bedeutet "oder"
s_arr = neu.split(" |e"); //= {"Gut", "n", "Morg", "n!"}
//reguläre Ausdrücke möglich -> später

//zerlegt den String an den angegebenen Stellen
//. hat eine Sonderbedeutung in RegEx, daher Escape notwendig
String str = "eins;zwei.drei,vier";
String[] t_arr = str.split(";|\\.|,");
for (int i = 0; i < t_arr.length; i++) {
    System.out.print(t_arr[i] + " "); //eins zwei drei vier
}

//Konvertierung Zahl2String
String s1 = String.valueOf(3.14);
//Konvertierung String2Zahl ... analog für alle primitive Datentypen, z.B. int
double pi = Double.parseDouble("3.14");
```



# Die Klasse StringBuilder

- Wenn eine Anwendung intensiv Zeichenketten manipulieren soll, dann nutzen Sie besser
  - StringBuilder
  - ein char []- Feld.

```
StringBuilder sb = new StringBuilder();  
sb.append("Habe nun, ach! Philosophie,\n");  
sb.append("Juristerei and Medizin,\n");  
sb.append("Und leider auch Theologie\n");  
System.out.print(sb.toString());
```





# Programmieren

2.3. Jar erzeugen



# Motivation

- Java-Programme bestehen aus vielen Klassen und verwenden viele Dateien (Konfigurationsdateien, Bilder, . . . )
  - Jede Klasse hat eine eigene `.class`-Datei
  - Programme sind über viele Verzeichnisse verteilt
  - Ausliefern: Zippen, versenden, dort wieder entpacken und starten  
Starten heißt: Der Kunde muss wissen, wo `main` liegt, und den classpath kennen
- Das muss einfacher gehen: `class`-Dateien in jar-Archiv zusammenfassen
  - JAR: Java ARchive
  - Im JDK enthalten ist ein Programm namens `jar` zum Anzeigen, Packen und Entpacken von jar-Dateien
  - jar-Dateien können direkt in den classpath eingebunden werden und müssen nicht vor der Verwendung entpackt werden



# Jar erzeugen in Eclipse

- JAR - Java Archive ist ein ZIP-Archiv, in dem zusätzlich Meta-Informationen in einer Datei META-INF/MANIFEST.MF enthalten sind.
- Wählen Sie in Eclipse das Projekt aus, das Sie exportieren wollen.  
File → Exportieren → Java → Runnable JAR file
- Namen des JAR Files festlegen <name>.jar
- Ggf. die Manifest-Datei auch im Projekt speichern. Dann Speicherort festlegen <pfad>/MANIFEST.MF
- Klasse angeben, die die main-Methode enthält.



# Jar erzeugen in Eclipse

- Aufruf mit  
`java -jar <name>.jar <args>`
- Ansehen des Inhalts  
`jar -tvf <name>.jar`
- Extrahieren von Dateien  
`jar -xvf <name>.jar`

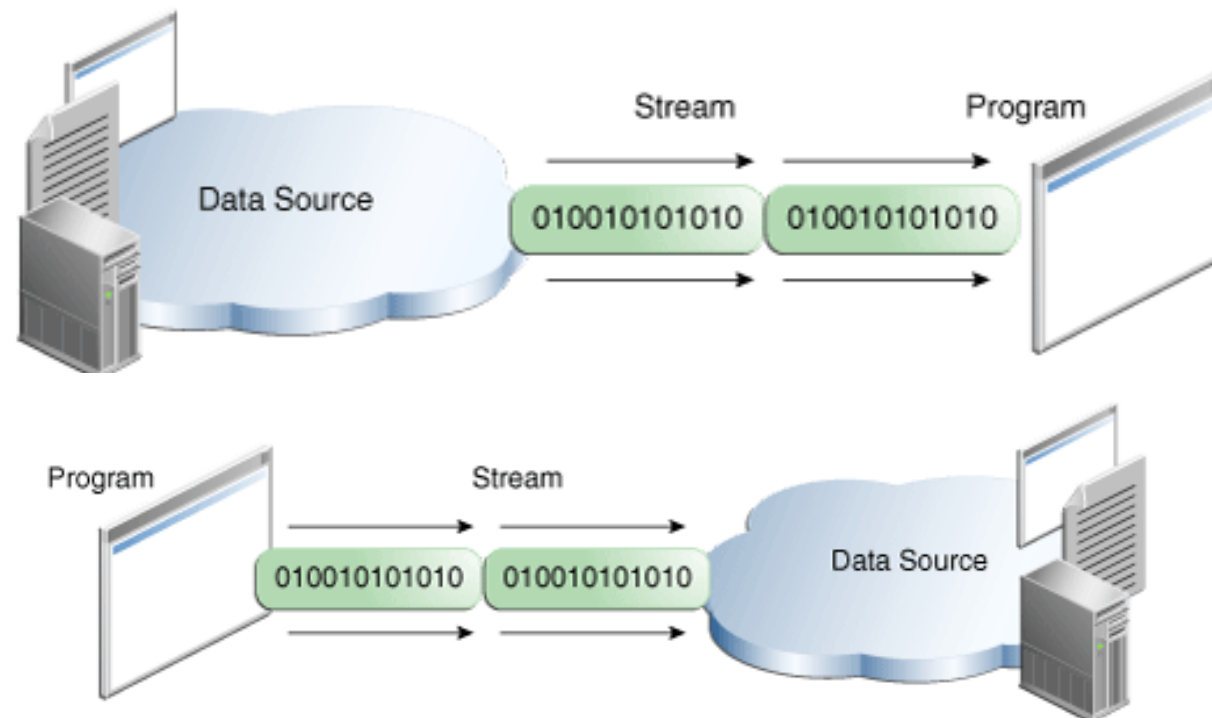


# Programmieren

## 2.4. I/O Grundlagen

# Streams

- Ein Stream (dt.: Strom) ist eine Datenstruktur, die Informationen sequentiell nach dem FIFO-Prinzip von einer Quelle zu einem Ziel befördert.
- Input-Stream und Output-Stream





# Streams

- Ein Stream, der Informationen von einer Quelle zu einem Java-Programm befördert, heißt **Input-Stream**.
- Ein Stream, der Informationen aus einem Java-Programm zu einer Datensenke (Ausgabegerät, Datei) befördert, heißt **Output-Stream**.
- Stream-Klassen sind im Paket `java.io` definiert.
- Das Java-Programm liest oder schreibt nicht direkt auf Datenquellen oder -senken, sondern von oder auf Streams.
- Die Arbeit mit externen Datenquellen oder -senken übernimmt das Betriebssystem.
  - Vorteil: Als Java-Entwickler muss ich mich nicht Betriebssystem-Details befassen.



# Datenquellen und –senken

- Interne Quellen bzw. Senken
  - byte-Arrays, char-Arrays
  - Strings
  - Pipes: Pipes dienen der Kommunikation zwischen Threads
- Externe Quellen bzw. Senken
  - Dateien
  - Geräte
  - Netzverbindungen
- Streams unterscheiden sich in Bezug auf
  - Art der Daten, die über den Stream transferiert werden
  - Ende-Verhalten (Dateien haben eine fixe Länge, die Tastatur nicht.)



# Streams: Datentypen

- **Binärdaten**
  - Daten liegen in einer internen Darstellung vor, z.B. Bilder, Tondateien, .exe, .class ...
- Für **Byte Streams** gibt es
  - zum Einlesen: `InputStream` oder eine Unterklasse
  - zum Schreiben: `OutputStream` oder eine Unterklasse
- **Textzeichen**
  - Daten liegen in einem Zeichensatz vor, z.B. .txt-Datei, .java, ... ,
- Für **Character Streams** gibt es
  - zum Einlesen: `Reader` oder eine Unterklasse
  - zum Schreiben `Writer` oder eine Unterklasse



# Benutzung von Streams

- Open
  - Stellt die Verbindung zu einer Datenquelle oder -senke her.
  - `FileReader fr = new FileReader(inFileName);`
  - `FileWriter fw = new FileWriter(outFileName);`
- Use
  - Lesen oder Schreiben der Daten.
  - `int ch = fr.read();`
  - `fw.write(ch);`
  - `read()` liest ein Zeichen, gibt ihn als `int` zurück. Wenn kein Zeichen mehr gelesen werden kann, dann liefert `read()` `-1`.
- Close
  - Am Ende muss der Stream geschlossen werden.
  - `fr.close();`
  - Alternativ ab Java 8: `try with resources`



# Beispiel FileReader

```
public class FileReaderBeispiel {  
  
    public static void main(String[] args) {  
        Reader reader = null;  
        try {  
            reader = new FileReader("input.txt");  
            int c = -1;  
            while ( (c = reader.read()) != -1) {  
                System.out.print((char)c);  
            }  
        } catch (IOException e) {  
            System.err.println("Fehler beim Lesen der Datei!");  
        } finally {  
            try { reader.close(); } catch (IOException e) {}  
        }  
    }  
}
```





# BufferedReader

- Nur mit dem `FileReader` über `read()` Dateien einzulesen ist umständlich, da man zeichenweise durch die Datei durchgehen muss.
- Um das Einlesen einfacher zu gestalten, gibt es die Klasse `BufferedReader`. `BufferedReader` ermöglicht es, zeilenweise mit `readLine()` durch eine Datei zu gehen oder auch eine vorgegebene Anzahl von Zeichen zu lesen.
- Wir können dann mit Hilfe eines `StringBuilder` den Inhalt der gesamten Datei in Portionen einlesen und weiterverwenden.
- Jede `line` ist ein normaler `String`.



# Beispiel BufferedReader

```
public class BufferedReaderBeispiel {  
  
    public static void main(String[] args) {  
        BufferedReader br = null;  
        try {  
            FileReader fr = new FileReader("input.txt");  
            br = new BufferedReader(fr);  
            StringBuilder builder = new StringBuilder();  
            String line = "";  
            while ((line = br.readLine()) != null) {  
                builder.append(line).append(System.lineSeparator());  
            }  
            System.out.println(builder.toString());  
        } catch (IOException e) {  
            System.err.println("Fehler beim Lesen der Datei");  
        } finally {  
            try { br.close(); } catch (IOException e) { }  
        }  
    }  
}
```





# Dateien schreiben

- Zum Schreiben zum Dateien gibt es das Pendant `BufferedWriter`.
- Dieser wird ähnlich wie der `BufferedReader` initialisiert. Das Beispiel ist einfach gehalten und beschränkt sich auf Standardanwendungen.

```
public class FileWriterBeispiel {  
  
    public static void main(String[] args) {  
        try {  
            FileWriter fr = new FileWriter("out.txt");  
            BufferedWriter br = new BufferedWriter(fr);  
            br.write("Wir schreiben ohne Umbruch ");  
            br.write("in die Datei");  
            br.newLine();  
            br.write("und dann den Umbruch");  
            System.out.println("done");  
            br.close();  
            fr.close();  
        } catch (IOException e) {}  
    }  
}
```



# Byte-Ströme lesen und schreiben

```
public class ByteStreamExample {  
  
    public static void main(String[] args) {  
        try {  
            InputStream in = new FileInputStream("input.txt");  
            OutputStream out = new FileOutputStream("output.txt");  
            byte[] buffer = new byte[in.available()];  
            in.read(buffer, 0, buffer.length);  
            for (int i = 0; i < buffer.length; i++) {  
                buffer[i]++;  
            }  
            out.write(buffer, 0, buffer.length);  
            in.close();  
            out.close();  
        } catch (IOException e) {}  
    }  
}
```





# I/O Streams – Komplexe Anwendungen

- Die vorgestellten Klassen funktionieren für einfache Anwendungsfälle.
- Es gibt weitere Pakete und Klassen für komplexere Szenarien
- Dazu gehören:
  - Angabe von relativen oder absoluten Pfadangaben in Filename, auch unabhängig vom Betriebssystem
  - Vor- und Nachbearbeitung der Daten des Streams, z.B. Komprimieren oder Verschlüsseln
  - Einstellen der Zeichen-Codierung: UTF-8, ISO-8859-1,...
- Für die korrekte Benutzung lesen Sie die API-Spezifikation und recherchieren Sie online.